# Beach Hero Architecture Hardening Memo

## Summary

Beach Hero already has a workable foundation: ScriptableObject-driven levels, prefab pools, a map progression screen, powerups, skins, store, ads, tutorials, and custom editor tooling. The main architecture gap is that the system boundaries are implicit. Core loop, meta loop, UI flow, economy, ads, saves, and gameplay entities call each other directly through global singletons.

The goal is to make Beach Hero more expandable and senior-level without breaking the current game. The recommended path is not a rewrite. It is a staged hardening plan: first stabilize existing behavior, then introduce clear boundaries around progression, economy, gameplay flow, UI, saves, ads, and content spawning.

## Current System Problems And Resolutions

### 1. Global Singleton Coupling

Problem: most systems call `GameController.GetInstance`, `UIController.GetInstance`, `AdController.GetInstance`, `SaveSystem`, and other global accessors directly. This makes changes risky because gameplay, UI, ads, saves, and economy all know about each other.

**Resolution:**

- Keep existing singletons during the transition.
- Add facade services first: `ProgressionService`, `EconomyService`, `PowerupService`, `AdService`, and `GameFlowService`.
- Existing controllers delegate to services.
- New code talks to services, not raw controllers.
- Reduce direct `GetInstance` calls subsystem by subsystem.

**Task memo:**

- Add service interfaces/classes without changing behavior.
- Route one vertical slice first: win rewards and next-level flow.
- Replace direct calls only after tests prove parity.

### 2. UI Owns Game Decisions

Problem: result tabs, main menu, gameplay UI, store UI, and map UI directly change game state, grant rewards, show ads, start levels, and save data.

**Resolution:**

- UI should become presentation plus user intent.
- UI sends commands like `RetryLevel`, `ContinueToNextLevel`, `ClaimWinReward`, `OpenStore`, and `StartSelectedLevel`.
- `GameFlowService` decides loading screens, ads, state transitions, map sync, and screen changes.

**Task memo:**

- Create `GameCommand` methods on `GameFlowService` .
- Move result-tab reward granting out of UI.
- Move map-to-game transition into flow service.
- Keep current screens visually unchanged.

## 3. Meta Loop Is Spread Out

Problem: level unlocks, stars, currency, boats, powerups, ads, rate-us, tutorials, and map state are scattered across `LevelDatabaseSO` , `MapController` , `StoreManager` , `SkinController` , `PowerupController` , and UI tabs.

**Resolution:**

- Create one `PlayerProgress` runtime model.
- Persist it through a typed save repository.
- `MetaProgressionService` owns level/stars/unlocks.
- `EconomyService` owns coins and purchases.
- `PowerupService` owns balances/unlocks/selection.
- Map and UI render this state instead of mutating it.

**Task memo:**

- Define `PlayerProgress` from existing save keys.
- Add migration/load from current ES3 keys.
- Keep writing old keys temporarily if needed for rollback.
- Add tests for default progress, level completion, stars, and currency.

## 4. Rewards Can Be Granted Multiple Times

Problem: win/lose tabs grant coins during `Open()` . Reopening a result screen can duplicate currency. Rewarded ad multiply also lives in UI.

**Resolution:**

- Create `LevelRunResult` .
- Create `RewardBundle` .
- Add `ClaimLevelReward(result, rewardMode)` with idempotency.
- UI displays reward state but does not directly mutate balance.

**Task memo:**

- Add reward claim state per completed run.
- Move base win/fail coin grant into `EconomyService` .
- Move rewarded multiplier into `AdService` and `EconomyService` .
- Add tests: result reopen does not duplicate rewards.

## 5. Game Flow Is Not Centralized

Problem: retry, next level, skip, home, map selection, and start gameplay each perform their own combination of loading screen, game state, level setup, UI screen opening, ad logic, and map animation.

**Resolution:**

- Introduce one flow layer:
- `BootGame`
- `OpenMainMenu`
- `OpenMap`
- `StartLevel`
- `RetryLevel`
- `CompleteLevel`
- `FailLevel`
- `ContinueAfterWin`
- `SkipAfterAd`
- `ReturnHome`
- Keep `GameController` initially, but make it delegate to this flow.

**Task memo:**

- First wrap existing calls without changing internals.
- Replace callers gradually.
- Add debug logs for every flow transition.
- Add play-mode smoke test for boot -> map -> start -> retry -> home.

## 6. `LevelController` Has Too Many Responsibilities

Problem: `LevelController` handles input, path drawing, player setup, level spawning, pooling, obstacle updates, collectables, star calculation, win/fail counters, and runtime state.

**Resolution:**

- Split responsibility over time:
- `LevelRunController` : current run state.
- `PathDrawingController` : draw path input and smoothing.
- `LevelSpawner` : level content spawn/despawn.
- `LevelResultCalculator` : stars and completion result.
- `LevelSimulationController` : player, obstacles, and collectables update loop.

**Task memo:**

- Do not split everything at once.
- Extract pure result calculation first.
- Extract spawn/despawn next because it is switch-heavy but contained.

- Leave path drawing last because it is gameplay-sensitive.

## 7. Switch-Heavy Content Expansion

Problem: adding a new obstacle, collectable, or powerup requires enum edits and switch edits in multiple files.

**Resolution:**

- Add ScriptableObject registries:
- `ObstacleRegistry`
- `CollectableRegistry`
- `PowerupRegistry`
- `RewardRegistry`
- Each registry maps type/id to prefab, pool, behavior, unlock, UI icon, and config.
- Existing enum values can remain at first.

**Task memo:**

- Start with collectables or obstacles, not both.
- Register existing types.
- Update spawner to ask registry for factory/pool.
- Add validation for missing registry entries.

## 8. Save System Is String-Based

Problem: save keys are static strings spread across systems. There is no typed save model, no versioning, and no migration strategy.

**Resolution:**

- Wrap ES3 behind `IProgressRepository` .
- Store/load a typed `PlayerProgress` .
- Support migration from existing keys.
- Keep current keys stable until migration is verified.

**Task memo:**

- Add read-only migration first.
- Add tests for old-key compatibility.
- Add save version field.
- Add debug dump tool for current progress state.

## 9. Ads Are Mixed With Gameplay/UI Decisions

Problem: ad policy lives inside `AdController` , result tabs, and flow methods. Rewarded ad callbacks mutate economy directly.

**Resolution:**

- `AdController` remains the SDK wrapper.
- `AdService` owns policy:
- when banners show
- when interstitials are eligible
- rewarded ad purpose
- no-ads state
- UI asks for "rewarded multiply" or "rewarded skip"; the service returns the result.

**Task memo:**

- Fix existing interstitial readiness bug first.
- Add `RewardedAdPurpose` enum.
- Route skip/multiply through `AdService` .
- Add fallback behavior for unavailable ads.

### 10. Boot Order Is Fragile

Problem: `Initializer` manually initializes many systems with async delays and direct singleton access. Failure handling is minimal.

**Resolution:**

- Introduce `GameBootstrapper` .
- Boot phases:
- local save load
- config defaults
- core services
- UI loading
- scene load
- remote/ads/IAP async services
- first screen
- Non-critical services should fail gracefully.

**Task memo:**

- Keep current `Init` scene.
- Wrap existing initialization order.
- Add logs per boot phase.
- Do not block game start on ads/IAP/remote config unless required.

## Concrete Task Roadmap

### Phase 0: Safety Net

- Add architecture notes and flow diagrams for current boot, core loop, and meta loop.
- Add smoke checklist for manual testing.
- Add logs for boot, level start, win, fail, reward claim, next, retry, skip, and home.
- Fix isolated bugs:
- total stars save bug
- interstitial readiness logic
- UI lambda unsubscribe issues
- screen stack close behavior

## Phase 1: Stabilize Progression And Rewards

- Add `LevelRunResult` .
- Add `RewardBundle` .
- Add `LevelResultCalculator` .
- Move coin/star reward decisions out of result tabs.
- Ensure win/fail rewards are claim-once.
- Keep UI behavior identical.

**Acceptance:**

- Winning a level grants expected coins/stars once.
- Reopening result UI does not duplicate coins.
- Retry does not advance level.
- Skip advances only after rewarded ad success.
- Next level path still opens map and animates correctly.

## Phase 2: Typed Progress Model

- Add `PlayerProgress` .
- Add `IProgressRepository` backed by ES3.
- Load existing save keys into typed progress.
- Route highest level, stars, coins, boat selection, powerups, no-ads, tutorial flags through progress services.
- Keep old keys compatible during transition.

**Acceptance:**

- Existing players keep their progress.
- Fresh install gets current defaults.
- Store, header UI, map, skins, and powerups show the same values as before.
- Progress can be dumped/debugged from an editor menu.

## Phase 3: Flow Service

- Add `GameFlowService` .
- Move main menu -> map -> gameplay -> result -> map/home transitions into it.
- UI buttons call flow commands.
- `GameController` becomes a compatibility facade while callers migrate.

**Acceptance:**

- All current buttons still work.
- Loading screen behavior is consistent.
- Game state transitions are logged and valid.
- No UI class directly calls level setup, reward grant, or ad policy.

### Phase 4: Content Registries

- Add registry for collectables and obstacles.
- Move pool selection out of `LevelController` switches.
- Add validation for missing prefab/pool/type mappings.
- Keep existing level data format for now.

**Acceptance:**

- Existing 100 levels spawn unchanged.
- New collectable/obstacle can be added through registry plus prefab/config.
- Invalid registry setup produces editor validation error before runtime.

### Phase 5: Service Boundaries And Cleanup

- Convert `StoreManager` , `SkinController` , `PowerupController` , and `AdController` into SDK/data adapters behind services.
- Reduce direct `GetInstance` usage in gameplay entities.
- Replace direct UI screen mutations with events/commands.
- Add tests around progression, economy, reward, and flow.

**Acceptance:**

- Core gameplay entities do not know about UI screens.
- UI screens do not know ES3 keys or reward math.
- Ads/IAP failures do not break core gameplay.
- New features can be added by touching fewer files.

## Non-Breaking Migration Rules

- Never refactor all controllers in one pass.
- Preserve existing scenes, prefabs, ScriptableObjects, and save keys until replacements are verified.
- Add new services as wrappers first; only move logic after behavior is covered.

- Use vertical slices: one behavior fully migrated and tested before the next.
- Keep old public methods temporarily as compatibility shims.
- Prefer additive changes over renames/moves early.
- Run manual smoke after every phase:
- boot
- main menu
- map
- start level
- draw path
- win
- next level
- fail
- retry
- skip with ad
- store purchase path
- boat customization
- settings
- no internet/ad unavailable path

## Definition Of Proper Architecture

- Core gameplay loop is owned by a run/session layer.
- Meta progression is owned by a progress/economy layer.
- UI sends intent, not business logic.
- Ads/IAP/remote config are adapters, not gameplay dependencies.
- ScriptableObjects author content; runtime state lives in typed progress models.
- New content is registry/config driven.
- Saves are typed and migration-safe.
- Flow transitions are centralized and testable.
- Existing gameplay remains unchanged while internals become safer.

## First Recommended Implementation Batch

- Fix total stars calculation.
- Fix interstitial readiness check.
- Fix unsafe UI event unsubscriptions.
- Add `LevelRunResult` , `RewardBundle` , and `LevelResultCalculator` .

- Move win/fail coin reward granting out of `GameWinTab` and `GameLoseTab` .
- Add claim-once protection for result rewards.
- Add simple edit-mode tests for stars/rewards/progression.

This batch gives immediate stability and creates the first clean architecture seam without requiring scene or prefab rewiring.